ThriveLens: Human-Confirmed Multimodal Food Analysis and Offline-First Fitness Intelligence with Parameter-Efficient Vision-Language Adaptation

Thabang Masoabi Tinotenda Muzembe David Seola

Chisomo Blessings Moleni Michael Bulis

Bachelor of Engineering — Artificial Intelligence and Machine Learning

Abstract—ThriveLens is a fitness and nutrition platform designed to connect multimodal food analysis, nutrition records, workout execution, progress monitoring, location-based discovery, and grounded coaching through a shared user history. This article presents a documentary systems study and a reproducible experimental protocol, integrating the supplied research blueprint with primary literature and technical documentation. The central design separates model-generated food candidates from confirmed dietary records: users confirm food identity and portion, a traceable catalogue supplies nutrient values, deterministic arithmetic constructs the meal, and subsequent actions reference saved evidence. The proposed learning experiment applies supervised quantized low-rank adaptation to Food-R1 using a frozen four-bit backbone, rank-64 language adapters, two NVIDIA T4 GPUs, and three fixed epochs. Documentary analysis identifies simulated application workflows, unresolved authorization and persistence requirements, and conflicting dataset archive identifiers. Arithmetic verification confirms that the declared training, validation, and test partitions total 302,157 records, whereas attributed source counts total 295,177, leaving 6,980 records unresolved. The evaluation protocol combines task-specific model metrics, calibration and abstention, paired nutrition assessment, offline recovery, authorization tests, and end-to-end device acceptance. Published Food-R1 findings are treated solely as external context; no new ThriveLens training, accuracy, latency, or user-study outcomes are asserted. The contribution is an auditable integration framework and an execution-ready evaluation specification that make the boundary between recognition, confirmation, computation, and durable action explicit.

Keywords—vision-language models, food analysis, human confirmation, QLoRA, offline-first systems, nutrition provenance, reproducibility, fitness intelligence

I. Introduction

Food recognition is only one step in maintaining a useful dietary record. The ThriveLens project, also named Omni Fitness AI in its source documents, addresses a broader workflow: identifying a food candidate, resolving its meaning and portion, recording nutrients, completing exercise, reviewing progress, planning meals, discovering nearby facilities, and obtaining contextual coaching. Its proposed contribution is therefore not another isolated food classifier. It is the preservation of evidence across decisions that otherwise become disconnected between application screens, model responses, and user records [1], [2].

A photograph is an observation, not a complete nutritional measurement. Visually similar dishes may differ in preparation, ingredients, or serving mass, and the supplied research explicitly treats hidden oils, allergens, and medical suitability as beyond reliable image-only determination. This motivates a confirmation boundary rather than automatic conversion of generated text into nutritional facts. The same concern applies downstream: an attractive progress chart is not evidence of completed exercise, and a confident coaching response is not evidence that its proposed foods, quantities, or actions exist in the application's records [1].

ThriveLens is consequently framed as a consumer self-management system, not a diagnostic service. Its design combines machine assistance with user correction, attributable data, deterministic calculations, and permission-bounded actions. Human oversight is not assumed to eliminate error; instead, it creates an explicit opportunity to correct observations and to retain the basis of the resulting record. This limited, accountable role is consistent with the emphasis on autonomy, safety, transparency, and stakeholder responsibility in the World Health Organization's guidance for health-related artificial intelligence [3], [4].

The study makes four connected contributions. First, it synthesizes the supplied implementation audit into a traceable account of what is documented, what is only configured, and what remains unmeasured. Second, it specifies an integrated architecture for human-confirmed nutrition, durable workouts, offline persistence, authenticated synchronization, and grounded coaching. Third, it consolidates the Food-R1 adaptation contract and checks the internal consistency of the declared dataset. Fourth, it defines a prospective evaluation spanning model quality, numerical nutrition error, reliability, privacy boundaries, and complete user tasks. The contribution is a systems design and documentary analysis with an executable research specification; it is not a report of a completed clinical or machine-learning trial.

The remainder of the article reviews the technical foundations, explains the documentary method and baseline, develops the target architecture and workflows, specifies data and training procedures, and presents the evaluation protocol. Results distinguish independently checked arithmetic from documentary findings and externally published benchmarks. The discussion then addresses validity, responsible deployment, and the artifacts required to support stronger empirical claims.

II. Background and Related Work

A. From Classification to Structured Food Understanding

Food-image research spans classification, ingredient recognition, and nutritional estimation. Food-101 offers a category-recognition benchmark, while Nutrition5k provides dish-level visual and nutritional information, including ingredient and mass-related measurements. These resources support different questions: correct category prediction does not, by itself, demonstrate ingredient completeness, portion accuracy, or nutrient validity. ThriveLens therefore retains task-specific evaluation instead of treating all food-related outputs as interchangeable measures of accuracy [5], [6].

The proposed project corpus also draws on Open Images and Fruits-360. Open Images supports diverse image annotations, whereas Fruits-360 is a fruit-and-vegetable recognition resource. Their inclusion may expand available visual supervision, but the meaning of each converted conversation still depends on its original annotation and acquisition process. The project's record totals are counts of its stated derived records, not necessarily the sizes of the original public datasets. Food2K is a relevant large-scale external food-recognition resource, but it is not established as part of the supplied ThriveLens corpus and must remain an optional external evaluation source [7], [8], [9].

Food-R1 is the selected public food vision-language checkpoint. Its original investigators describe multi-task supervised training followed by reinforcement learning, and report recognition, ingredient, and numerical food benchmarks. The public release is associated with the Qwen3-VL model family. ThriveLens's proposed experiment starts from that released checkpoint and applies supervised QLoRA adaptation; it does not reproduce the original Food-R1 reinforcement-learning procedure or establish a new foundation model [10], [11].

B. Parameter-Efficient Adaptation

Low-rank adaptation, or LoRA, updates a frozen weight matrix through trainable low-rank factors. It offers an alternative to optimizing every model parameter and makes the adaptation artifact separable from the base model. QLoRA extends this approach by training adapters over a quantized backbone, including NormalFloat four-bit representation and double quantization. These methods motivate the proposed constrained-memory experiment, but their published feasibility does not establish that the particular ThriveLens software stack will execute successfully without a compatibility test [12], [13].

ZeRO reduces redundant training-state storage across distributed workers. The project selects ZeRO stage 2, two workers, and gradient accumulation to support a defined effective batch size. These are complementary configuration choices rather than independent guarantees of speed, memory fit, or convergence. The research protocol therefore requires a two-device smoke run, measured resource telemetry, verified adapter updates, and a restart test before committing to the full experiment [1], [14].

A legacy MobileNetV2 route provides a separate image-classification baseline. MobileNetV2 is relevant to efficient mobile vision, but a category classifier does not have the same output contract as a food vision-language model. Any comparison must restrict both systems to compatible labels and identical eligible examples. A TensorFlow Lite export, even when available, would establish neither Food-R1 deployment on a phone nor end-to-end nutrition validity [2], [15].

C. Confidence, Persistence, and Accountable Decisions

Confidence should be evaluated against observed correctness rather than accepted from fluent generated language. Temperature scaling is a useful calibration method when meaningful scores are available, but it must be fitted without using the final test set. Calibration is also task-dependent: a well-calibrated food label cannot reveal an unobserved ingredient. ThriveLens therefore combines calibrated decision thresholds, abstention, confirmation, and source-backed calculation instead of using a single confidence number as permission to log a meal [16].

For a longitudinal application, reproducibility extends beyond a trained adapter. Dataset identity, preprocessing, model revision, inference settings, persistence semantics, and failure handling all affect what a user sees and saves. The emphasis on recorded configuration, accessible experimental artifacts, and transparent reporting follows the broader reproducibility principles discussed by Pineau et al. The present study applies those principles to the complete chain from model input to saved activity and authorized downstream action [17].

III. Study Method and Documentary Baseline

A. Source Scope and Evidence Classification

The primary project evidence comprises the supplied ThriveLens 2026 Research Blueprint and the supplied research synthesis bearing the full ThriveLens title. The blueprint anchors its repository audit to commit 244b4bbe79b4e563 df94af13affbe552d2f189b0, dated 8 September 2026, in the reported repository bulismichael/-thrivelens-2026. The application root is reported as omni-fitness-ai/. This article analyzes those documents; it does not claim an independent checkout, code execution, dataset download, or device audit of that repository [1], [2].

The method has four steps. Project statements are first classified as reported implementation observations, stated configuration, target requirements, or measured outcomes. Cross-document statements are then checked for agreement, and unresolved contradictions are retained rather than silently reconciled. Declared record counts are independently recomputed. Finally, primary publications and official documentation are used to contextualize the design and refine the prospective evaluation. Additional recommendations derived in this article are identified as design or validity considerations, not as historical implementation facts.

Table I summarizes the resulting evidence categories. A configuration is evidence of an intended procedure, not evidence that the procedure completed. Likewise, a dependency manifest supports the presence of a declared package, not functioning authentication, persistence, inference, or native integration. No experiment registry, executed training outputs, participant dataset, test report, or deployment telemetry was supplied. The

protocol is therefore prespecified in this article, but is not described as externally preregistered.

TABLE I
EVIDENCE CLASSES USED IN THIS STUDY

Evidence class	Interpretation and boundary
Documentary finding	An implementation or configuration statement attributed to the supplied project audit; not independently executed here.
Checked arithmetic	Sums, differences, and percentages recalculated from the declared counts; not a raw-data audit.
External benchmark	An outcome reported by another study, under its own data and evaluation conditions.
Target / protocol	A proposed architecture, hypothesis, test, or release requirement; no success is implied.

Project evidence: [1], [2].

B. Reported Application State

The supplied audit describes a substantial interface prototype with approximately 92 repository tree items and 63 text-like files. Its reported manifest includes Expo SDK 57, React Native 0.86.3, React 19.2.3, Expo Router, NativeWind, Drizzle, expo-sqlite, and Supabase libraries. Image selection, notifications, secure storage, and authentication-session packages are listed, while camera, location, and native-map dependencies required by the target design are not established in that snapshot. These are historical observations attributed to the project documents, not claims about the latest package releases [1], [2].

The feature audit reports timer-based authentication and artificial-intelligence responses, no persisted session guard, local-only onboarding state, static dashboard values, and a scanner displaying sample foods rather than a live inference result. Workout controls do not establish durable completed-set records, and navigation away from the session can lose state. Nutrition and planning controls are partly inactive; maps contain demonstration points and directions; two coach routes duplicate functionality. The exercise catalogue is reported to contain one JSON record, with referenced media absent. These conditions support an interface-prototype description, not a production-readiness claim [1].

The most consequential security finding is the reported SQL schema that disables row-level security on the listed application tables and includes a password_hash field in an application-level users table. The target design replaces this arrangement with Supabase Auth as the identity authority, application profiles linked to authenticated identities, and row-level policies on exposed data. Official Supabase documentation supports this separation and emphasizes authorization for exposed tables. Because the present work does not execute the schema, it reports the documented risk without asserting a successful exploit or completed remediation [2], [18], [19].

C. Research Questions and Hypotheses

RQ1 asks whether supervised QLoRA improves structured food understanding over the unadapted Food-R1 checkpoint on the same eligible project examples. RQ2 asks whether user confirmation and canonical nutrition lookup reduce dietary-record error relative to direct model estimates. RQ3 asks whether local persistence and synchronization preserve intended records through disconnection, restart, replay, conflict, and account changes. RQ4 asks whether the fixed two-T4 experiment is feasible and reproducible under measured resource constraints. RQ5 asks whether grounded, typed, approval-gated coaching reduces unsupported statements and invalid or duplicate actions [1], [2].

The corresponding hypotheses are prospective. H1 predicts better prespecified task performance after adaptation. H2 predicts lower nutrient error after confirmation and source-backed computation. H3 predicts that atomic local writes, an outbox, idempotency, and explicit version handling preserve records without duplicate business effects. H4 predicts fewer unsupported or unauthorized actions when coaching is constrained to validated evidence and approved commands. None is treated as confirmed by the existence of the design. Failure criteria and measurement denominators are specified in Section VIII so that an unsuccessful run or integration test remains reportable evidence.

IV. System Architecture and Data Contracts

A. Separation of Responsibilities

The target system separates six responsibilities: presentation, domain logic, local persistence, authenticated services, model inference, and external data providers. Screens render state and collect input; typed domain services validate quantities and transitions; local repositories store durable records; authenticated services authorize synchronization and provider access; the model proposes bounded observations; and external adapters resolve canonical food or location data. This organization is a target architecture synthesized from the project sources, not a claim that all six layers are already implemented [1].

Figure 1 presents the central decision sequence. Image-based inference yields a candidate, with uncertainty and limitations, rather than a committed food log. Manual search and barcode lookup can enter through catalogue resolution without requiring vision inference. The user chooses or corrects the food and portion before deterministic calculation. The stored meal then becomes evidence for summaries and coaching. A direct model-to-meal database write is deliberately excluded because it would bypass the project's defining confirmation and provenance boundary.

The word offline-first refers to local continuity, not universal offline intelligence. Manual logging, saved workouts, previously stored records, and appropriately cached catalogue data should remain usable without connectivity. Fresh remote inference, uncached provider searches, and cloud synchronization may be deferred. The application must label these states explicitly and avoid displaying a stale response as a new measurement. No on-device deployment of the released Food-R1 checkpoint is demonstrated by the supplied evidence [1], [2].

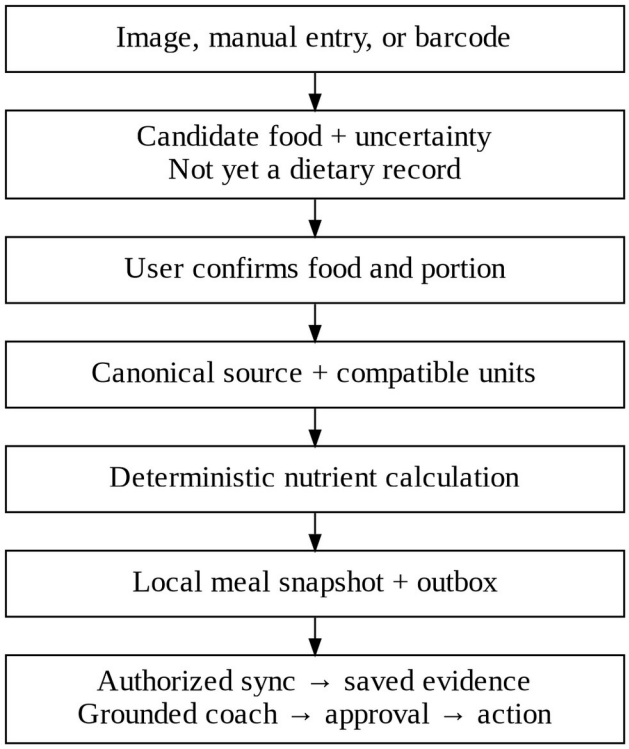


Fig. 1. Target human-confirmed nutrition and action pipeline, synthesized from the project blueprint [1]. Manual and barcode routes may resolve a catalogue candidate without vision inference.

B. Identity and Authorization

Authentication establishes a server-verifiable identity. Domain tables use a stable user identifier linked to that authority, while the server derives ownership from the validated session rather than trusting a user_id supplied in a request body. Client-side navigation guards improve usability but are not authorization controls. Policies must enforce access to both directly owned records and child records whose ownership is inherited through a parent, including meal components, workout sets, and conversation actions [18], [19].

Session tokens require secure device storage, controlled refresh, and explicit expiry handling. Provider secrets and privileged database credentials belong on trusted services. In particular, the earlier suggestion of exposing a model-provider token through an EXPO_PUBLIC variable is not suitable for a secret: Expo documents that public environment variables are embedded in client code. The serving contract must therefore keep such credentials outside the distributable application and use authenticated server mediation [2], [20].

Account switching is a data-isolation event rather than only a screen transition. The design partitions local records by account, hides the previous account's cached data, cancels or rebinds account-scoped reminders, removes obsolete tokens, and prevents queued mutations from being submitted under another identity. A queued operation retains its original owner and is never silently reassigned. Private food photographs use protected storage, narrowly scoped access, defined retention, and deletion linked to the user's data controls.

C. Local Persistence and Synchronization

SQLite provides a durable local store, but SQLite availability does not itself implement synchronization. The target design uses typed repositories over a versioned schema and a mutation outbox. A domain update and its corresponding outbox entry should commit atomically in one local transaction. Each operation includes an operation identifier, entity identifier, owner, mutation type, base version, payload, timestamps, and retry state. This transactional elaboration makes the blueprint's durability requirement explicit [1], [21].

Figure 2 specifies the intended write lifecycle. The server authenticates the request, checks ownership and version compatibility, and deduplicates the operation identifier in the same transaction as the accepted domain effect. An acknowledged response allows the client to mark the operation complete. A lost acknowledgment triggers a retry with the same identifier, not a newly invented mutation. This targets exactly-once business effects under retried delivery; it does not claim that the network delivers a message exactly once.

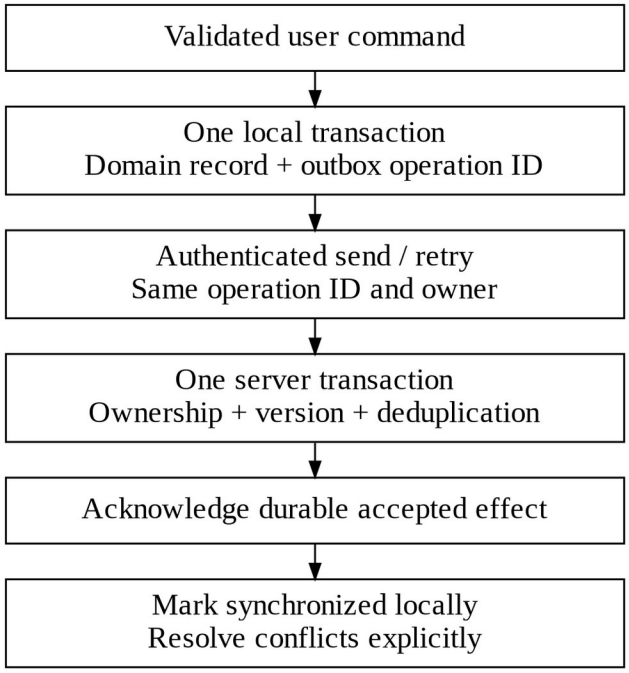


Fig. 2. Target mutation lifecycle. Retried delivery retains one operation identity; the design targets exactly-once accepted business effects, not exactly-once network delivery [1].

Conflict handling depends on domain meaning. Field-aware last-write-wins resolution may be acceptable for low-risk preferences when timestamps and ownership are valid. It is not an adequate universal rule for meals, completed sets, or measurements, where silent overwriting could erase activity or alter summaries. Those records should favor append-oriented events or explicit corrections, version checks, and visible conflict handling. Tombstones preserve deletion intent until synchronization completes, with retention and cleanup rules documented separately.

D. Longitudinal Data Model

The identity layer stores profile preferences, units, locale, time zone, consent, onboarding state, and the user-entered information required for selected fitness estimates. Food-source and catalogue entities retain provider identifiers, barcode or search identity, language, nutrient basis, units, freshness, and missing-value status. A meal record retains candidate and confirmed food identifiers, portion and conversion basis, an immutable nutrient snapshot, date, provenance, model revision when relevant, and any correction history. This prevents a later provider update from silently rewriting previously logged intake [1].

Planning entities include recipes, recipe ingredients, pantry quantities, meal-plan items, and shopping-list items. Exercise entities store stable identifiers, aliases, equipment, body region, difficulty, instructions, reviewed form cues, and media provenance. Workout plans remain separate from sessions and completed sets; planned repetitions must not be counted as performed activity. Progress records distinguish measured, estimated, and unavailable values. Places include provider identity, coordinates, type, address, retrieval time, and nullable hours or ratings rather than fabricated defaults.

Reminders retain time zone, consent state, delivery identifiers, and account-scoped device tokens. Conversation records retain authorized source identifiers, messages, proposed actions, validation results, approval state, and model revision. Operational entities include the synchronization outbox, consent history, export and deletion jobs, and minimal audit events. These contracts support traceability without requiring every potentially sensitive field to be exposed to every service.

V. Integrated Nutrition, Fitness, and Coaching Workflows

A. Human-Confirmed Nutrition

The food workflow accepts an image, manual query, or barcode and returns an editable candidate. The model should identify uncertainty and request clarification when the visible evidence is insufficient. Confirmation requires both a food identity and a portion representation; it is not satisfied by a generic acceptance button over an opaque calorie total. A user may replace the candidate, correct an ingredient, choose a serving unit, or abandon the result. An unresolved item remains unresolved rather than being converted into a zero-valued meal [1], [2].

Provider adapters resolve the confirmed item to a canonical record. The source design names USDA FoodData Central and Open Food Facts as potential nutrition providers. Their coverage, response structure, attribution, and permitted reuse must be handled independently. FoodData Central offers structured API access; Open Food Facts has its own API and licensing requirements. Provider selection should therefore retain the specific source and retrieval metadata instead of merging records into an untraceable generic food database [22], [23], [24].

For nutrient k, the deterministic calculation uses the confirmed quantity and a compatible catalogue basis:

$$N_k = \sum_{i=1}^{M} \frac{m_i}{b_i} c_{ik} \tag{1}$$

Here, M is the number of confirmed components, m_i is the confirmed mass of component i, b_i is the mass basis of the catalogue value, and c_{ik} is the amount of nutrient k at that basis. The ratio is valid only when the units and conversion are compatible. A serving-based record requires a supported serving-to-mass or equivalent serving conversion; a volume is not converted to mass without an appropriate density or product-specific relationship. Internal calculations retain precision, while display rounding occurs only at the presentation boundary.

Energy is represented in kilocalories and nutrient quantities in explicit units. Unknown nutrient values are not treated as zero, and incomplete nutrient totals carry a visible completeness status. A saved snapshot includes the provider record and portion assumptions so that later review can distinguish arithmetic correctness from input uncertainty. Catalogue-backed calculation improves traceability but cannot make a wrongly selected food or estimated portion accurate by definition.

Onboarding-derived basal-metabolic-rate or total-daily-energy-expenditure targets remain transparent, editable estimates. The supplied sources do not establish a single implemented equation and coefficient set, so this article does not insert one as a project fact. Before release, the selected method must be versioned, its required inputs disclosed, and its unit conversions and boundary cases covered by known-answer tests. These estimates must not be presented as measured metabolism or individualized medical advice.

B. Meal Planning, Pantry, and Shopping

Meal proposals reference canonical food and recipe identifiers. The application validates quantities, nutrient completeness, and totals before displaying a plan for approval. Pantry matching subtracts compatible quantities and normalizes equivalent units before generating shopping items. Duplicate ingredients are consolidated only when identity and conversion are justified; ambiguous product matches or unsupported conversions require user review. This keeps a language model from creating apparently precise shopping quantities from inconsistent underlying units [1].

Plans remain distinct from consumption. Approving a plan does not automatically log that the meal was eaten, and generating a shopping list does not imply availability at a nearby store. Changes to pantry inventory, planned servings, and completed meals have separate state transitions and provenance. Such separation is necessary for truthful daily summaries and for evaluating whether the system supports an actual user workflow rather than only producing plausible recommendations.

C. Workout Execution and Progress

A workout session moves through planned, active, paused, completed, or abandoned states. Each completed set is persisted before the rest timer begins, with actual repetitions, load, unit, completion time, and optional perceived exertion. A durable rest deadline permits recovery after app backgrounding or restart. Adding a set creates a record rather than only changing a visual

counter. Completing a session reconciles its recorded sets and does not infer completion from elapsed screen time [1].

The exercise-library target is at least 300 reviewed entries across chest, shoulders, biceps, triceps, back, quadriceps, hamstrings, abdominals, and gluteal muscles. This is a delivery requirement, not a verified content count. Each entry requires an identifier, equipment requirements, instructions, safety-oriented form cues, provenance, and a usable media fallback. Unsupported "1,500+ exercises" claims should not be carried forward from interface copy when the documented catalogue does not substantiate them [1], [2].

Progress summaries derive from saved activity and explicit time-zone boundaries. Training volume, where appropriate for a recorded exercise, is calculated from completed repetitions and load after validated unit conversion; planned sets are excluded. Charts distinguish measured body weight from estimates and missing data. Progression suggestions use transparent, bounded rules over recent completed sessions, remain editable, and require approval. They are not injury assessments, diagnoses, or substitutes for qualified exercise guidance.

D. Places, Directions, and Reminders

The proposed location stack combines native MapLibre rendering, device location, MapTiler geocoding or tiles, OpenStreetMap-derived place queries, and PostGIS-backed spatial filtering. These components have different roles: a renderer does not supply a complete place catalogue, and a map pin does not establish current business details. MapLibre's Expo integration requires an appropriate native development build rather than an assumption that the feature will run in Expo Go [25], [26], [27].

Place results retain source and retrieval time, and the interface allows manual town search when location permission is denied. Cached results must identify freshness, while directions hand off to a supported device navigation application. Unknown opening hours, ratings, prices, stock, or store coverage remain unknown. OpenStreetMap attribution and service-use policies apply to the relevant data and infrastructure; public map or query services must not be treated as unrestricted production backends [28], [29].

Reminders are opt-in, time-zone aware, and reversible. The application records notification identifiers to update or cancel scheduled events, avoids cross-account delivery, and continues to function when permission is denied. Neither the presence of a notifications package nor a successful scheduling call proves real delivery under every device state. Permission denial, time-zone changes, logout, expired device tokens, and background behavior therefore belong to the native acceptance tests.

E. Grounded Coaching and Accessible Interaction

A single canonical coaching route retrieves only data authorized for the active account. Requests contain the minimum relevant history, including saved meals, completed workouts, preferences, and approved catalogue identifiers. Responses separate explanatory text from typed action proposals and cite the records supporting user-specific statements. Assertions about today's intake, for example, must derive from today's saved records rather than from a generic conversational estimate [1].

Action validation checks identifier existence, ownership, units, numerical totals, allowed state transitions, and source references. A user approves a proposed change before an idempotent command is executed. The model does not emit executable SQL, obtain privileged credentials, or directly mutate records. Retrieved notes and external text are treated as untrusted content, not instructions that can override system permissions. Timeout, refusal, malformed output, missing evidence, and partial provider failure produce explicit non-destructive outcomes.

Presentation preserves the project's visual identity while replacing demonstration constants with repository-backed state. Loading, empty, offline, error, and permission-denied states must be distinguishable. Labels, focus order, readable text scaling, contrast, reduced motion, keyboard behavior, and touch targets are part of acceptance, as are text alternatives for charts. These are target quality requirements; the supplied documents contain no completed accessibility audit. Their inclusion matters because a confirmation control that users cannot understand or operate does not constitute meaningful oversight.

VI. Dataset Definition and Integrity

A. Declared Partitions and Source Reconciliation

The blueprint declares 242,756 training records, 6,460 validation records, and 52,941 test records. Their sum is exactly 302,157. Table II reports these counts and percentages calculated against the declared total. The final test set is intended to remain untouched during adaptation, hyperparameter screening, checkpoint decisions, and calibration [1].

TABLE II
DECLARED PROJECT PARTITIONS

Partition	Records	Share (%)
Training	242,756	80.34
Validation	6,460	2.14
Test	52,941	17.52
Total	**302,157**	**100.00**

Source counts: [1]. Shares are calculated against 302,157 and rounded to two decimal places.

The stated source counts do not reconcile to the same total. Open Images contributes 107,794 records, Fruits-360 contributes 183,893, and Nutrition5k contributes 3,490. Their sum is 295,177, leaving 6,980 records without attribution in the supplied counts. Table III makes this discrepancy visible. The residual is 2.31% of the declared total; it is a provenance gap, not evidence of an additional identified dataset. Without the raw manifest, this article cannot determine whether the mismatch reflects omitted sources, duplicates, transformation counts, or a documentation error [1], [2].

TABLE III
SOURCE ATTRIBUTION AND UNRESOLVED RESIDUAL

Source / status	Records	Share (%)
Open Images	107,794	35.67
Fruits-360	183,893	60.86
Nutrition5k	3,490	1.16
Attributed subtotal	295,177	97.69
Unattributed residual	6,980	2.31
Declared total	**302,157**	**100.00**

The residual is a documentation discrepancy, not an identified additional source [1], [2].

The blueprint also records two competing archive SHA-256 values. Both are reproduced in Appendix A with their source labels. Neither is promoted to a verified checksum because no corresponding archive was supplied for hashing. The executable experiment must freeze an archive name, byte count, member count, cryptographic digest, and manifest digest together. A filename or digest copied from a planning document is insufficient to establish the identity of the bytes used in training.

B. Record Schema and Validation

Each derived example should have a stable record identifier, source dataset, acquisition group, task label, media reference, conversation turns or prompt, image placeholders, target response, and split. Numerical targets include units and a documented interpretation. Ingredient targets require a consistent normalization rule, while structured responses require a versioned schema. A record converted into a conversation must retain the original annotation's limits; classification labels cannot silently become measured ingredient weights [1], [2].

Validation proceeds before model loading: archive integrity and member enumeration, manifest parsing, identifier uniqueness, source and task totals, media resolution, RGB decoding, conversation and image-token consistency, target validity, unit checks, and split leakage checks. Failed records are counted with reasons. Excluding a corrupt record changes the effective dataset and must be reflected in the frozen manifest, not hidden by a loader that skips failures at runtime.

The configured processor uses a minimum image-pixel budget of 65,536 and a maximum of 589,824, with sequence length capped at 2,048 tokens. These are pixel-budget and token limits, not proof that every image is resized to one square resolution. The experiment must use the pinned Food-R1 processor and tokenizer and report actual truncation or rejection behavior. No undocumented normalization, augmentation, or target repair should be inserted and then described as the original project configuration [1].

C. Group-Aware Partitions and Coverage

Related observations must remain in one partition. Potential grouping units include the original physical object or acquisition sequence for repeated fruit images, the original image for derived crops, and the dish for multiple Nutrition5k views. The precise grouping keys are not supplied, so the protocol requires their definition and audit before claiming leakage-resistant evaluation. Group-stratified splitting seeks class balance while preventing group overlap; it does not guarantee that every source or rare label can be perfectly balanced [30].

The stated corpus is strongly dominated by Fruits-360-derived records. Record volume therefore cannot establish representative coverage of mixed meals, regional cuisines, cooking methods, or naturally occurring portion distributions. Results must report source and task denominators, source-wise and task-wise metrics, and failure coverage rather than only a pooled score. Cuisine, geography, acquisition setting, and the selection rule for the 3,490 Nutrition5k records remain unspecified in the supplied evidence.

A further validity consideration follows from the reviewed Food-R1 literature: the upstream model already uses Nutrition5k-related supervision. Isolation between ThriveLens's own training and test partitions would therefore not prove that every test item was unseen during upstream model development. This is a potential overlap risk, not a demonstrated leakage finding. Where upstream identifiers are available, overlap should be audited; claims about genuinely unseen data should additionally use verified non-overlapping or newly collected evaluation material [10].

VII. Parameter-Efficient Adaptation and Training Protocol

A. Learning Objective and Adapter Design

The main experiment adapts the released zy12123/Food-R1 checkpoint through supervised next-token training on the frozen project conversations. For a base weight matrix W_0, the LoRA parameterization is:

$$W = W_0 + \frac{\alpha}{r} BA \tag{2}$$

The low-rank factors A and B are trainable, while the quantized base remains frozen. The configured rank r is 64 and scaling parameter α is 16. The training objective is the negative log-likelihood of eligible target tokens conditional on the image and prompt; prompt and padding treatment must be documented in the collator. This objective adapts the supplied checkpoint to the project response distribution without asserting that it reproduces Food-R1's original reinforcement-learning rewards [1], [12], [13].

The blueprint specifies all-linear language-side adapter targets with exclusions for lm_head, embed_tokens, and visual spans. The vision tower and merger are frozen; bias training and DoRA are disabled. Actual matched module names and the number of trainable parameters must be saved, since a configuration string alone cannot prove that the intended modules were selected. A smoke test must verify nonzero gradients and changed adapter weights without unintended updates to frozen components.

B. Fixed Main Configuration

Tables IV and V consolidate the principal experiment. The numerical configuration is retained from the source blueprint rather than replaced with a newly invented optimization recipe. Three epochs are mandatory for this identified baseline, validation uses all 6,460 declared validation records after each

epoch, and early stopping is disabled. The final-epoch adapter is the baseline evaluation artifact. Selecting an earlier checkpoint or changing the stopping rule would define a different experiment and must be identified accordingly [1].

TABLE IV
FIXED MAIN TRAINING CONFIGURATION

Parameter	Configured value
Hardware / workers	2 NVIDIA T4 GPUs; 2 distributed workers
Training length	3 epochs; early stopping disabled
Batch per GPU	1
Gradient accumulation	8 steps
Effective global batch	16
Optimizer	Paged AdamW, 8-bit
Learning rate / schedule	2×10^{-4}; cosine
Warmup / weight decay	0.05 / 0.01
Adam betas / epsilon	(0.9, 0.999) / 10^{-8}
Maximum gradient norm	0.3
Distribution	DeepSpeed ZeRO-2; NCCL
Validation	All 6,460 records after each epoch
Checkpoint / logging	Every 1,000 / 10 optimizer steps
Retention / telemetry	4 checkpoints / 60-second interval

Reported notebook configuration, not an executed result [1].

TABLE V
MODEL, ADAPTER, AND GENERATION CONTRACT

Component	Configured value
Base checkpoint	zy12123/Food-R1; exact revision to be recorded at execution
Quantization	4-bit NF4; double quantization enabled
Compute precision	FP16; BF16 and TF32 disabled
LoRA rank / alpha	64 / 16
LoRA dropout	0.05
Targets	Language-side linear modules; exclude output head, embeddings, and visual spans
Frozen modules	Vision tower and merger; quantized base
Bias / DoRA	None / disabled
Maximum sequence	2,048 tokens
Image pixel budget	65,536 minimum; 589,824 maximum
Gradient checkpointing	Enabled; use_reentrant=False
Final generation	Greedy; one beam; no sampling; up to 512 new tokens
Final test coverage	All 52,941 declared test records

Source: [1]. Actual revisions, matched modules, runtime compatibility, and completion require recorded execution evidence.

The effective batch size follows directly from the stated configuration:

$$B_{eff} = 1 \times 8 \times 2 = 16 \tag{3}$$

This is a configured global batch size, not a throughput measurement. Batch packing, sampler behavior, and a partial final batch affect realized update accounting, so the final run manifest must record optimizer steps and presented examples. The implementation must verify loss scaling under distributed accumulation rather than assume that combining the packages preserves the intended optimization semantics.

Input loading uses four workers, prefetch factor two, pinned memory, and persistent workers where supported. Archive-backed loading requires per-process archive handles, or an explicitly identified extracted-data path, to avoid unsafe handle sharing. Gradient checkpointing is configured with use_reentrant=False. DeepSpeed stage 2 runs with NCCL and two torchrun workers. These choices remain subject to compatibility and memory tests for the actual pinned runtime [1].

C. Execution Gates and Artifact Integrity

The source notebook is described as 30 cells, indexed 0–29. Its substantive stages include configuration and logging; runtime and archive checks; manifest, leakage, and image validation; dependency and CUDA checks; trainer and processor preparation; two-rank collective testing; a task-stratified smoke run; adapter and resume checks; fixed full training; loss and artifact inspection; full distributed evaluation; and a checksum-verified delivery archive. Appendix B preserves the cell-stage map and minimum run-manifest requirements [1].

The smoke run is a technical gate, not an accuracy result. It must establish that both GPUs participate, representative tasks produce finite losses, trainable parameters update, a checkpoint can be saved and reloaded, and generation uses the intended processor and output schema. A successful one-device example is not a substitute for the two-device test. Failures, environment patches, and compatibility workarounds must be recorded with the experiment, including any change that affects the training objective or optimizer state.

The full run saves every 1,000 optimizer steps, retains four checkpoints, logs every ten steps, and records resource telemetry at 60-second intervals. Checkpoint retention must not accidentally remove the final delivery artifact. Each validation pass records eligible, failed, and completed examples. Training duration, peak memory, utilization, throughput, restart behavior, and adapter checksums are outputs to be measured, not values inferred from the number or type of GPUs [1].

The project's 30-hour Kaggle planning allowance is a budget assumption, not a verified current platform entitlement. Feasibility must be estimated from the measured smoke-run rate, projected full validation and test generation costs, checkpoint overhead, and storage constraints. A resumed multi-session run remains acceptable only when its optimizer, scheduler, sampler, and random-state continuity are documented. A smaller completed experiment must be reported as smaller, not relabeled as the full baseline.

D. Controlled Comparisons and Budget Variants

The mandatory comparator is the unadapted Food-R1 checkpoint evaluated with the same quantization, processor, prompts, generation settings, eligible examples, and scoring

code as the adapted model. This isolates the adapter comparison more carefully than contrasting against a published score from another dataset or inference environment. The main test generation is greedy, uses one beam, does not sample, and permits up to 512 new tokens. In the source configuration, MAX_TEST_SAMPLES=0 denotes the entire 52,941-record test set, not an empty test [1].

The earlier budget variant uses a deterministic balanced 96,000-record manifest, 6,000 optimizer steps, rank 32, effective batch 16, maximum sequence length 1,536, task-aware image budgets in the stated 128–384 range, and 2,000 test generations. These settings jointly alter data, capacity, sequence budget, and evaluation coverage. It is therefore a distinct experiment, not a one-factor rank ablation or a completed substitute for the full test. Any direct comparison must state these confounders [1], [2].

Proposed follow-on comparisons include rank 32 versus 64, full versus balanced training data, and a separately identified limited-vision adaptation. Computationally affordable screening may use group-aware three-fold cross-validation restricted to training data; the validation set then supports the declared selection and calibration procedure. Candidate learning rates, dropout, and adapter scaling must be frozen before test access. Nested cross-validation is appropriate when estimating a complete selection procedure, but it is not assumed affordable for every large-model variant in the project's constrained budget [30], [31].

The legacy MobileNet route requires special care because the source descriptions conflict. The blueprint describes augmentation, a 128-unit dense layer, and several callbacks; the supplied research synthesis reports that the pinned code instead uses a simpler frozen-backbone pipeline with dropout and a short final-layer fine-tuning stage. This article cannot adjudicate that code-level disagreement without the exact executable artifact. A future baseline must identify its actual code hash and executed configuration and compare only compatible classification outcomes [1], [2].

VIII. Evaluation Protocol

A. Experimental Matrix and Test Governance

Table VI links each experiment to a claim. The final project test partition is reserved for the frozen evaluation, with no threshold tuning, prompt revision, checkpoint selection, or failure-driven label editing after inspection. Any post-test correction creates a new version and requires a transparent account of what changed. Primary tasks, secondary outcomes, eligible examples, rejection rules, confidence intervals, and comparison direction must be specified before execution [2].

TABLE VI
PROSPECTIVE EXPERIMENT MATRIX

ID	Comparison / scenario	Primary evidence
E0	Unadapted Food-R1	Frozen project-task baseline
E1	Rank-64 QLoRA vs. E0	Paired task metrics; full test
E2	Rank 32 vs. rank 64	Capacity / resource trade-off with other factors controlled
E3	Frozen vs. limited vision tuning	Separate optional visual-adaptation study
E4	Full vs. balanced training corpus	Data-volume / composition effects; confounders disclosed
E5	Direct estimate vs. confirmed catalogue workflow	Independent-reference nutrient error and interaction burden
E6	Offline, restart, replay, conflict	Record survival, duplicate effects, correct summaries
E7	Owner vs. other-account access	Positive and negative authorization tests
E8	Grounded typed coach vs. unconstrained proposal	Unsupported claims, validation failures, approved-action correctness

E2–E4 are follow-on comparisons, not completed experiments. E5–E8 evaluate the pipeline rather than neural accuracy alone [2].

A single aggregate score should not mix calorie error, ingredient F1, schema validity, and synchronization success without a predefined interpretation and weighting. The protocol instead emphasizes task-specific outcomes and a compact system scorecard. Macro-averaged source and task views complement pooled metrics when imbalance is substantial. Model-quality improvements are not used to waive data-integrity or security failures.

B. Structured and Numerical Model Metrics

Structured-output validity is the proportion of eligible generations satisfying the declared schema, including required fields and types. Semantic correctness is evaluated separately through exact match, normalized field match, token F1, ingredient precision, recall, F1, and intersection-over-union as appropriate. Classification top-1 accuracy and macro-F1 apply only to compatible single-label tasks. Normalization rules, synonym handling, duplicate ingredients, and empty-target conventions are fixed in the evaluator rather than adjusted after observing model errors.

For n eligible numerical predictions with reference y_i and estimate $\hat{y}_i$, mean absolute error and root mean squared error are:

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|\hat{y}_i - y_i\right| \tag{4}$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(\hat{y}_i - y_i\right)^2} \tag{5}$$

Errors are reported separately for energy, mass, and each nutrient in their natural units. Signed bias reveals systematic over- or underestimation that an absolute metric hides. Mean absolute percentage error uses an explicitly defined eligible set and zero-reference rule; arbitrarily inserting a small denominator

after seeing the outputs is not acceptable. The percentage MAE used in an external paper is not automatically the same quantity as MAPE. In particular, 100 × MAE divided by the mean reference value differs from the mean of per-example percentage errors.

Invalid output, service failure, truncation, and missing numerical fields remain visible. Schema-validity and completion denominators include all eligible examples. Numerical errors may be calculated on the declared measurable subset, but must be accompanied by coverage and failure counts so that a model cannot appear accurate merely by failing on difficult cases. A prespecified failure-aware utility can supplement this reporting; it must not silently replace the individual outcomes.

C. Calibration, Abstention, and Statistical Uncertainty

Calibration uses held-out validation data, never final test outcomes. When reliable class scores or another defensible correctness score exist, temperature scaling, reliability diagrams, expected calibration error, or an appropriate proper scoring rule can be evaluated. A model-generated sentence such as "high confidence" is not treated as a calibrated probability. Thresholds for asking the user, offering alternatives, or abstaining are frozen before final evaluation [16].

Coverage–risk analysis reports the fraction of examples for which the system answers and the error among those answers. An abstaining system must also report user burden and unresolved-task frequency, because low error at negligible coverage would not demonstrate practical usefulness. Food-identity confidence, ingredient uncertainty, and portion uncertainty remain distinguishable. In particular, certainty about a visible class does not justify certainty about an invisible cooking ingredient.

Comparisons use paired examples and acquisition-group-aware resampling. Ninety-five-percent confidence intervals should reflect the grouping structure rather than treat multiple views of one dish as independent observations. Each estimate reports its actual denominator, and multiple secondary comparisons are identified as exploratory or handled with a prespecified multiplicity strategy. Resampling uncertainty from one trained adapter does not capture training-seed variability; if only one full run is feasible, that limitation is stated rather than implying repeatability across seeds.

D. Paired Human-Confirmation Evaluation

The nutrition study compares direct model estimation with the confirmation-and-catalogue workflow on matched food cases and an independent reference. The reference should be established through documented composition and measured portions, or another defensible procedure that does not simply reuse the same estimated catalogue output as its own ground truth. A manual reference workflow may serve as an additional comparator. This design elaborates the blueprint's human-confirmation hypothesis without claiming that participants or measurements already exist [1], [2].

Outcomes include per-nutrient MAE, signed bias, correction frequency, portion edits, unresolved cases, completion time, and task abandonment. The paired error reduction is defined as direct-estimate error minus confirmed-workflow error, so a positive value favors the proposed pipeline. Analysis should distinguish food-identity correction from portion correction and, where feasible, compare measured-portion and user-estimated-portion conditions. Otherwise, the experiment could credit recognition for an improvement actually produced by weighing the food.

A human study requires a separately documented sample-size rationale, recruitment and inclusion criteria, consent, data handling, and any required ethical review before participation. The supplied material provides no participant count or variance estimate, so neither is invented here. A pilot can estimate variance and interaction burden to inform the powered study. Task order should be counterbalanced or randomized when the same participant uses multiple interfaces, and repeated observations from one participant should be modeled as related rather than independent.

E. Offline Reliability and Authorization

Persistence tests use deterministic fixtures that create, edit, and delete meals, workout sets, and preferences under network loss, process termination, restart, delayed acknowledgment, replay, concurrent version conflict, and account switching. The expected state is defined in advance from the accepted operations. Outcomes include surviving intended records, duplicate business effects, visible unresolved conflicts, incorrect daily totals, and synchronization completion. Zero duplicates is a target in these tests, not a result already obtained.

Two-account authorization tests cover owner-positive and other-user-negative access for reads, inserts, updates, and deletes on every owned table and relevant storage object. Tests also cover anonymous and expired sessions, forged client identifiers, child records attached to another user's parent, and privileged endpoints. Each test records expected and actual behavior. A zero-violation result in a finite suite would support that suite's coverage, not prove the absence of every possible vulnerability [2], [19].

Recovery tests verify that local domain writes and outbox creation are atomic, rejected operations remain explainable, and repeated acknowledgments do not duplicate summaries. Export and deletion checks include private images, local caches, queued operations, and documented retention exceptions. Backup restoration and schema migrations require compatibility tests because a restored database or upgraded app can otherwise reintroduce stale operations or break ownership relationships.

F. Serving, Native Devices, and End-to-End Acceptance

Serving evaluation records cold and warm latency distributions, including p50, p95, and p99 where sample size permits, throughput under stated concurrency, peak memory, timeout frequency, malformed responses, and model revision. Results specify hardware, software, input size, output limits, caching, and network conditions. Failure traces are redacted. Neither the Food-R1 literature nor a training-throughput estimate substitutes for mobile-to-service latency measurements.

Native acceptance covers image capture or selection, barcode flow, authentication and session recovery, location denial, map rendering, external directions, notifications,

accessibility, and background transitions on identified Android and iOS builds. The core daily loop comprises sign-up, persisted onboarding, manual or confirmed food logging, completed workout sets, correct daily summaries, and recovery after restart. Extended acceptance adds pantry-aware planning, shopping-list consolidation, place discovery, reminders, and an approved coach action that can be traced to saved sources [1].

Completion is recorded only when the intended durable state is observed, not when a button animates or a success toast appears. The release evidence should include a test identifier, build and service revisions, device and operating-system details, expected state, actual state, and reproducible failure evidence. The experimental protocol thus evaluates the full socio-technical pipeline rather than assuming that a model score establishes application reliability.

IX. Results

A. Documentary and Arithmetic Findings

The first result is the demonstrated mismatch between project intent and the implementation state reported in the supplied audit. Major interfaces exist, but the documented scanner, authentication, coaching, maps, and several logging interactions rely on simulation or static content. The SQL authorization design and missing persistence integration are material blockers because they affect ownership and the validity of longitudinal records. These findings are documentary results, not newly executed penetration or device tests [1], [2].

The second result is a reproducible arithmetic check. The split equation is 242,756 + 6,460 + 52,941 = 302,157. The source equation is 107,794 + 183,893 + 3,490 = 295,177. Their difference is 6,980, equal to approximately 2.31% of the declared total. Thus, split totals are internally consistent while source attribution is incomplete. This result does not establish that 6,980 files are missing; only a frozen, inspected manifest could resolve that interpretation.

The third result concerns artifact identity. Two different expected archive digests occur in the supplied evidence, and the descriptions of the legacy MobileNet pipeline also disagree. These discrepancies prevent treating the documents as a self-sufficient record of an executed experiment. The result of the audit is therefore a set of concrete evidence requirements: authoritative data identity, code and environment revisions, executed logs, adapter artifacts, complete predictions, and test records tied to the target application state.

B. External Food-R1 Benchmark Context

Table VII reports selected outcomes from the original Food-R1 investigators. They are included to describe the capability claims associated with the chosen upstream checkpoint, not as ThriveLens measurements or as directly comparable scores for the project corpus. The tasks, source datasets, evaluation procedures, and numerical units differ, so the values should not be combined into a single score or transferred to the proposed QLoRA adapter [10].

TABLE VII
SELECTED PUBLISHED FOOD-R1 RESULTS—EXTERNAL ONLY

Benchmark	Metric	Reported value
Food-101	Top-1 accuracy	95.21%
VireoFood-172	Ingredient IoU / F1	78.50 / 84.61
CalorieBench	Calorie MAE	42.56 kcal
CalorieBench	Calorie RMSE / R^2	112.89 kcal / 0.81
Nutrition5k	Calorie MAE / pMAE	27.6 kcal / 14.4%
Nutrition5k	Fat / carb. / protein MAE	3.2 / 4.4 / 4.0 g

Reported by Zhu et al., Tables 1, 3, and 5 [10]. IoU and F1 retain the source's reporting scale. These are not ThriveLens measurements; pMAE is not interchangeable with MAPE.

The distinction is especially important for calorie estimation: a benchmark outcome does not validate a particular user's portion, hidden ingredients, or saved meal. Likewise, a published classification score does not establish schema validity or source-backed logging in the mobile application. The required project comparison remains unadapted and adapted Food-R1 evaluated under the same frozen ThriveLens protocol.

C. Status of ThriveLens-Specific Empirical Outcomes

The supplied blueprint records no saved execution outputs from the Food-R1 notebook at its audit point. No complete project prediction file, adapter evaluation, serving benchmark, native-device report, human-confirmation trial, or acceptance-test result was supplied. Table VIII records the evidence status instead of filling a conventional results table with invented values. "Not measured in supplied evidence" is not equivalent to zero performance, test failure, or impossibility; it identifies the boundary of what this article can establish [1], [2].

TABLE VIII
THRIVELENS-SPECIFIC EVIDENCE STATUS

Outcome family	Status in supplied evidence
Dataset arithmetic	Checked here; source attribution remains unresolved.
Repository behavior	Reported by the supplied audit; not newly executed here.
QLoRA training / adapters	Configured; no completed-run outputs supplied.
Held-out model metrics	No project predictions or full test results supplied.
Human-confirmed nutrient error	Paired study specified; no participant results supplied.
Serving performance	No measured latency, memory, or throughput supplied.
Offline / RLS / device tests	Acceptance protocol specified; no completed reports supplied.

A missing measurement is not a measured zero or a demonstrated failure [1], [2].

The resulting conclusion is narrower than a validated product claim but scientifically meaningful: the proposed system has a coherent, testable evidence pipeline, while the documented prototype and data description do not yet substantiate its performance or release readiness. The protocol makes those

claims falsifiable by specifying the observations required to support or reject them.

X. Discussion and Threats to Validity

A. Contribution and Trade-offs

ThriveLens's strongest contribution is the explicit transition from uncertain observation to confirmed, source-attributed, durable action. This separation allows recognition, portion entry, catalogue matching, arithmetic, synchronization, and coaching to fail visibly and to be evaluated independently. It also supports explanations anchored in saved records: a coach can identify which meal or workout informed a suggestion rather than merely produce a plausible narrative. These are architectural properties and design arguments, not measured improvements in the present study.

The approach introduces costs. Confirmation adds interaction time, catalogue matching can remain ambiguous, and offline operation requires conflict handling and visible freshness information. Restricting model actions may reduce conversational convenience, while provider-backed calculation depends on data coverage and compatible units. The relevant question is therefore not whether oversight is free, but whether the measured reduction in error and unsafe action justifies its burden for the intended tasks. The paired study and end-to-end acceptance protocol are designed to expose that trade-off.

A modular design can also improve diagnosability. An incorrect food label, wrong serving conversion, duplicate synchronization effect, and unsupported coaching statement should have different error records and remediation paths. Treating all four as a single "AI accuracy" problem would conceal the source of harm. Conversely, the presence of modules is not sufficient: their contracts must be enforced under retries, failures, account changes, and evolving model versions.

B. Data and Model Validity

The source-count discrepancy, unresolved archive digest, unknown grouping keys, unspecified task distribution, and strong source imbalance limit confidence in the training corpus before any model is fitted. A large declared dataset cannot compensate for unclear provenance or repeated acquisitions crossing partitions. The possible overlap with upstream Food-R1 training adds a separate generalization concern. These limitations require a data audit and, where appropriate, genuinely independent evaluation rather than optimistic interpretation of a pooled test score.

Freezing vision components may reduce adaptation cost but constrain visual domain adaptation. Increasing adapter rank may improve capacity while increasing overfitting risk. Additional supervised training may improve the project response format yet degrade an upstream capability. The fixed three-epoch protocol should therefore retain all validation trajectories and disclose regressions. Reporting the final checkpoint does not require pretending that every task improved; a negative or mixed outcome is a legitimate result of the declared experiment.

Comparability also depends on inference conditions. Processor revisions, image budgets, sequence truncation, quantization, prompts, and scoring normalization can materially alter outputs. A baseline using one configuration and an adapter using another would confound attribution. The reproducibility manifest and identical evaluation contract are designed to make those differences inspectable rather than hide them behind the same model name [2], [17].

C. Systems and Human Validity

A local database can preserve a write while a flawed server summary still double-counts it. A row-level policy can correctly protect a parent table while a related storage object remains exposed. A native feature can work in a foreground development session but fail after token expiry or account switching. The evaluation therefore includes negative tests and state recovery rather than only successful demonstrations. No finite suite establishes universal correctness, so test coverage and observed failure boundaries must accompany any release claim.

Human confirmation may introduce its own errors, including agreement with an authoritative-looking suggestion, mistaken portion entry, or abandonment caused by friction. The interface must make correction meaningful, but the current documents do not provide usability or automation-bias measurements. Cultural and linguistic coverage are also unestablished. A worldwide product ambition should therefore not be represented as validated effectiveness across cuisines, countries, literacy levels, or accessibility needs [1], [4].

The manuscript's own limitation is documentary dependence. It consolidates supplied audits and independently checks their numerical consistency, but it does not newly inspect source code, reconcile raw files, execute GPUs, recruit participants, or run the app. The evidence classification prevents this limitation from being confused with a failed experiment. It also explains why external model results and detailed configurations cannot support a claim that the proposed system has already achieved its objectives.

XI. Responsible Deployment and Release Governance

A. Safety, Consent, and Data Stewardship

The intended scope is consumer fitness and nutrition assistance. The system must not claim to detect allergens from a photograph, diagnose illness, prescribe treatment, assess injury, or guarantee medical suitability. Risk-sensitive prompts should receive bounded information and appropriate direction toward qualified support rather than extreme dietary or exercise instructions. The combination of health-adjacent inputs and generative output requires explicit limits and oversight consistent with WHO guidance [3], [4].

Food photographs, location history, body measurements, and conversation context require purpose-limited handling. Collection should be minimized, permissions separated by function, and retention explained. User export and deletion should cover domain records, private media, caches, and pending synchronization operations, with any backup-retention behavior documented. Model prompts and telemetry should exclude unnecessary identifying details, and production logs should not become an uncontrolled duplicate store of sensitive user data.

Licensing is an independent release gate. Permission to use a model checkpoint does not automatically establish rights to every training image, food database, map tile, exercise video, or

derived redistribution. Food-source provenance and applicable attribution must survive caching and transformation. Open Food Facts' data and image terms require separate consideration, and the source dataset terms must be checked for the exact downloaded versions and intended use [1], [24].

No completed human-participant study, institutional ethics approval, funding declaration, or conflict-of-interest statement is established by the supplied evidence. This article does not invent such declarations. Before a venue requires them or a participant study begins, the research team must supply accurate author-approved statements and obtain the applicable review. The listed supervisor is acknowledged as documented supervision, not presented as an ethics authority or an unverified coauthor.

B. Release Gates and Workstream Dependencies

Release readiness is defined by evidence rather than screen completion. Table IX consolidates the blueprint's dependency-aware gates. The foundation gate establishes identity, authorization, persistence, and native feasibility. The daily-loop gate proves that saved nutrition and exercise produce correct summaries. Extended capabilities then add planning, locations, reminders, and controlled coaching. Model-enabled release finally requires full evaluation, real image integration, repeatable builds, and operational recovery [1].

TABLE IX
EVIDENCE-GATED RELEASE SEQUENCE

Gate	Required evidence
G1 Foundation	Real authentication and sessions; two-account RLS tests; versioned API and schema; SQLite/outbox recovery; native-map feasibility.
G2 Daily loop	Persisted onboarding; manual food logging; durable completed sets; restart recovery; correct date- and unit-aware summaries.
G3 Integrated features	Pantry-aware plans; deduplicated shopping list; sourced places and directions; reminders; validated and approved coach actions.
G4 Model release	Real image-to-confirmed-log integration; reconciled data; complete held-out report; reproducible builds; privacy-aware operations and rollback.

Targets synthesized from the blueprint; none is marked passed without its corresponding test evidence [1].

The source workstreams allocate platform and data foundations to A, nutrition and scanning to B, workouts and progress to C, places and reminders to D, AI services and release controls to E, and continuing training and evaluation to T. These are responsibility slots, not verified assignments to particular authors. Shared identity, schemas, API contracts, and migration ownership precede dependent feature integration; model experimentation can proceed in parallel only after data-integrity gates are satisfied.

The planning logic is sequential where correctness depends on shared contracts and parallel where activities are genuinely independent. A relative schedule may group foundation and data reconciliation first, model and native preparation second, controlled experiments and integration third, and final evaluation and acceptance last. The documents do not establish completed dates for those phases. A pull request, test result, migration, acceptance record, and runbook together define completion more meaningfully than a roadmap checkbox.

C. Reproducibility and Operational Control

Each release requires an immutable application revision, dependency lock, environment example without secrets, migration history, and build instructions. Model artifacts require the exact base and processor revisions, adapter files and hashes, dataset and split manifests, executed configurations, logs, full predictions, failure lists, evaluation code, calibration artifacts where applicable, and a model card. Appendix B specifies the minimum fields that allow another investigator to distinguish a rerun from a differently configured experiment [1], [17].

Operational controls include authenticated serving, rate and size limits, explicit timeout behavior, redacted observability, backup restoration, model-version rollback, and revocation of compromised credentials. A model update must not silently alter old meal snapshots, and a provider refresh must not rewrite recorded history without an explicit correction. Risk review should cover the entire lifecycle, not only pre-release scoring; the NIST AI Risk Management Framework and its generative-AI profile provide relevant voluntary structures for that continuing process [32], [33].

XII. Conclusion

This article develops ThriveLens as an evidence-preserving fitness and nutrition system rather than an autonomous source of dietary truth. Its central workflow separates multimodal observation, user confirmation, canonical nutrient lookup, deterministic calculation, durable local records, authenticated synchronization, and approved coaching actions. The supplied project research supports a detailed architecture and a fixed supervised QLoRA experiment, while its audit describes an interface prototype with substantial integration and authorization requirements still unresolved.

The documentary and arithmetic results are concrete: the declared split counts reconcile to 302,157 records, the attributed sources account for 295,177, the residual is 6,980 records, and competing archive digests prevent definitive artifact identification from the documents alone. Published Food-R1 outcomes contextualize the selected checkpoint but do not establish ThriveLens performance. The proposed evaluation addresses that distinction through controlled adapter comparisons, source-aware model metrics, independent paired nutrition assessment, offline recovery, authorization checks, and complete device workflows.

The defensible contribution is a coherent systems design, an explicit audit of its evidence, and a reproducible path for testing its claims. A stronger claim—that ThriveLens improves nutritional accuracy or reliably supports everyday fitness practice—requires the corresponding executed evidence. Preserving this boundary makes the research useful both when the proposed methods succeed and when they reveal limitations that require redesign.

Acknowledgment

The authors acknowledge Sant Kumar Maurya (E13548) for supervising the ThriveLens project [1].

Appendix A. Project Provenance and Unresolved Identifiers

A. Author and Audit Record

The documented author roster is Thabang Masoabi (23BAI70376), Tinotenda Muzembe (23BAI70589), David Seola (23BAI70652), Chisomo Blessings Moleni (23BAI70696), and Michael Bulis (23BAI70620). The reported audit anchor is repository bulismichael/-thrivelens-2026, application directory omni-fitness-ai/, commit 244b4bbe79b4e563df94af13affbe552d2f189b0, dated 8 September 2026. These identifiers are reproduced as documentary provenance, not evidence of an independently executed repository inspection [1].

B. Conflicting Archive Digests

The legacy summary digest recorded in the blueprint is:

```
6c5320da2316117583e82516fa443198
6b4dc36117efec8aafa9e0779e68961e
```

The notebook's expected digest recorded in the blueprint is:

```
4c378fc53d427fff808a2bfee7d923d5
569355fd3794b5ffd5bfc3a4115d7f49
```

Both strings have the form of 64-character SHA-256 values, but they identify different candidate artifacts. The authoritative digest can be established only by hashing the exact archive selected for the experiment and reconciling its metadata and manifest. Line wrapping here is typographic and does not add characters to either digest [1].

C. Legacy Baseline Disagreement

The blueprint describes a MobileNetV2 pipeline with random flip, rotation, and zoom, a 128-unit dense head, and ModelCheckpoint, EarlyStopping, and ReduceLROnPlateau. The supplied research synthesis instead reports ImageNet initialization, 224 × 224 inputs, a directory-based 80/20 split with seed 123, a frozen backbone, global average pooling, dropout 0.2, a softmax head, and a subsequent two-epoch fine-tuning stage for the last 30 layers at learning rate 10^{-5}. It also reports that the additional augmentation and callback logic is absent in the pinned code [1], [2].

These accounts must not be blended into a fictitious executed configuration. A defensible legacy comparison requires the exact code snapshot, split identities, trained checkpoint, class mapping, and outputs. Its early-stopping description also does not modify the separate Food-R1 baseline, which retains three fixed epochs with no early stopping.

Appendix B. Reproducibility Record and Artifact Delivery

A. Notebook Stage Map

Table X maps the reported 30-cell notebook to its execution evidence. Cells 0, 8, 18, 23, and 27 are described as markdown separators. The remaining cells contain the configuration, checks, training, inspection, evaluation, and delivery stages. This map is a source-reported specification; it is not an execution log [1].

TABLE X
REPORTED NOTEBOOK EXECUTION STAGES

Cells	Stage / required artifact
1–2	Logger and fixed configuration
3–4	Runtime inspection and archive identity
5–7	Manifest, schema, leakage, and RGB checks
9–10	Pinned dependencies; CUDA and package checks
11–12	Trainer fingerprint; model and processor preparation
13–14	Two-rank NCCL collective test; launch command
15–16	Task-stratified smoke run; adapter verification
17	Resume compatibility
19–20	Fixed full training; run manifest
21–22	Loss, gradients, and adapter-file inspection
24–26	Full distributed evaluation; predictions, failures, metrics
28–29	Delivery archive, checksum, and final checks

Cells 0, 8, 18, 23, and 27 are markdown separators in the reported notebook [1].

B. Minimum Run Manifest

A completed run records experiment identity, application and training-code commits, notebook checksum, start and end times in UTC, and the selected hypothesis or comparison. Data fields include archive name, byte and member counts, archive checksum, manifest checksum, total and effective split sizes, source and task distributions, grouping rules, rejected-record reasons, and split identifiers. These fields must refer to the actual bytes consumed rather than to an earlier planning inventory.

Model fields identify the base checkpoint and revision, processor and tokenizer revisions, matched adapter modules, frozen components, trainable parameter count, rank, alpha, dropout, quantization type, double-quantization flag, and compute dtype. Optimization fields include optimizer, learning rate, scheduler, warmup, betas, epsilon, weight decay, clipping, microbatch size, accumulation, world size, effective batch, requested and completed epochs, and optimizer-step counts.

Runtime fields include random seeds, operating system, Python, PyTorch, CUDA, Transformers, PEFT, bitsandbytes, DeepSpeed, GPU identity and memory, distributed backend, checkpointing mode, worker settings, and any compatibility patches. Preprocessing fields include image budgets, sequence limits, conversation template, token masking, image handling, and truncation behavior. Evaluation fields include generation settings, scorer revision, eligible denominators, prediction and failure-file checksums, task metrics, confidence intervals, and validation-fitted calibration parameters when applicable.

Artifact fields identify retained checkpoints, final adapter checksum, validation history, full test prediction file, metric outputs, failure list, logs, telemetry, environment lock, model card, and delivery-archive checksum. A resumed run additionally records the checkpoint, optimizer and scheduler restoration, sampler continuity, and any interrupted intervals. Missing artifacts are declared; a final success message does not replace them.

C. Application and Release Package

The application package contains the immutable source revision, dependency lock, safe environment example, build and deployment instructions, authentication configuration, migrations, row-level policies, local schema, seeds, and repository and outbox tests. Nutrition deliverables include provider adapters, attribution, unit and arithmetic fixtures, editable confirmation screens, and provenance snapshots. Fitness deliverables include the reviewed exercise catalogue, media rights and fallbacks, durable session logic, and progress fixtures [1].

Location and reminder deliverables include native-build configuration, permission and denial behavior, place-source attribution, freshness and caching rules, spatial queries, direction handoff, notification cancellation, and account isolation. AI deliverables include the authenticated inference contract, model and calibration revisions, schema validators, evidence-grounded coach actions, approval and replay controls, and adversarial-input tests. Release evidence includes device, accessibility, security, end-to-end, backup-restoration, and rollback records, alongside a privacy-aware operations runbook.

References

[1] ThriveLens project team, "ThriveLens 2026 Research Blueprint: Omni Fitness AI," unpublished project document, 2026, 23 pp. Supplied source; repository audit anchor: Sep. 8, 2026.

[2] ThriveLens project research materials, "ThriveLens: Human-Confirmed Multimodal Food Analysis and Offline-First Fitness Intelligence with Parameter-Efficient Vision-Language Adaptation," unpublished research synthesis, 2026, 34 pp. Supplied source.

[3] World Health Organization, Ethics and Governance of Artificial Intelligence for Health. Geneva, Switzerland: WHO, 2021, ISBN 978-92-4-002920-0.

[4] World Health Organization, Ethics and Governance of Artificial Intelligence for Health: Guidance on Large Multi-Modal Models. Geneva, Switzerland: WHO, 2024, ISBN 978-92-4-008475-9.

[5] L. Bossard, M. Guillaumin, and L. Van Gool, "Food-101—Mining discriminative components with random forests," in Proc. European Conf. Computer Vision (ECCV), 2014, pp. 446–461.

[6] Q. Thames et al., "Nutrition5k: Towards automatic nutritional understanding of generic food," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2021. arXiv:2103.03375.

[7] A. Kuznetsova et al., "The Open Images Dataset V4: Unified image classification, object detection, and visual relationship detection at scale," Int. J. Comput. Vis., vol. 128, pp. 1956–1981, 2020, doi: 10.1007/s11263-020-01316-z.

[8] Fruits-360, "Fruits-360 datasets," official dataset project. Accessed: Sep. 10, 2026. [Online]. Available: https://github.com/fruits-360

[9] W. Min et al., "Large scale visual food recognition," arXiv:2103.16107, 2021, revised 2023, doi: 10.48550/arXiv.2103.16107.

[10] Y. Zhu et al., "Food-R1: A unified multi-task food vision-language model with reinforcement learning," arXiv:2606.04986v1, Jun. 3, 2026, doi: 10.48550/arXiv.2606.04986. Preprint.

[11] zy12123, "Food-R1," Hugging Face model card and checkpoint repository. Accessed: Sep. 10, 2026. [Online]. Available: https://huggingface.co/zy12123/Food-R1

[12] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," arXiv:2106.09685, 2021, doi: 10.48550/arXiv.2106.09685.

[13] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," arXiv:2305.14314, 2023, doi: 10.48550/arXiv.2305.14314.

[14] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory optimizations toward training trillion parameter models," in Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC), 2020, pp. 1–16, doi: 10.1109/SC41405.2020.00024.

[15] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510–4520, doi: 10.1109/CVPR.2018.00474.

[16] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. Int. Conf. Machine Learning, PMLR, vol. 70, 2017, pp. 1321–1330.

[17] J. Pineau et al., "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," J. Mach. Learn. Res., vol. 22, no. 164, pp. 1–20, 2021.

[18] Supabase, "Managing user data," Supabase Auth documentation. Accessed: Sep. 10, 2026. [Online]. Available: https://supabase.com/docs/guides/auth/managing-user-data

[19] Supabase, "Row level security," database documentation. Accessed: Sep. 10, 2026. [Online]. Available: https://supabase.com/docs/guides/database/postgres/row-level-security

[20] Expo, "Environment variables in Expo," documentation. Accessed: Sep. 10, 2026. [Online]. Available: https://docs.expo.dev/guides/environment-variables/

[21] Expo, "SQLite," Expo SDK documentation. Accessed: Sep. 10, 2026. [Online]. Available: https://docs.expo.dev/versions/latest/sdk/sqlite/

[22] U.S. Department of Agriculture, Agricultural Research Service, "FoodData Central API guide." Accessed: Sep. 10, 2026. [Online]. Available: https://fdc.nal.usda.gov/api-guide/

[23] Open Food Facts, "Open Food Facts API documentation." Accessed: Sep. 10, 2026. [Online]. Available: https://openfoodfacts.github.io/openfoodfacts-server/api/

[24] Open Food Facts, "License: Be on the legal side," API documentation. Accessed: Sep. 10, 2026. [Online]. Available: https://openfoodfacts.github.io/documentation/docs/Product-Opener/api/tutorials/license-be-on-the-legal-side/

[25] MapLibre, "Expo," MapLibre React Native setup documentation. Accessed: Sep. 10, 2026. [Online]. Available: https://maplibre.org/maplibre-react-native/docs/setup/expo/

[26] MapTiler, "Geocoding API," MapTiler Cloud documentation. Accessed: Sep. 10, 2026. [Online]. Available: https://docs.maptiler.com/cloud/api/geocoding/

[27] Supabase, "PostGIS: Geo queries," database extensions documentation. Accessed: Sep. 10, 2026. [Online]. Available: https://supabase.com/docs/guides/database/extensions/postgis

[28] OpenStreetMap Foundation, "Tile usage policy," Operations Working Group. Accessed: Sep. 10, 2026. [Online]. Available: https://operations.osmfoundation.org/policies/tiles/

[29] OpenStreetMap contributors, "Overpass API," OpenStreetMap Wiki. Accessed: Sep. 10, 2026. [Online]. Available: https://wiki.openstreetmap.org/wiki/Overpass_API

[30] scikit-learn developers, "StratifiedGroupKFold," API reference. Accessed: Sep. 10, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html

[31] scikit-learn developers, "Nested versus non-nested cross-validation," model-selection example. Accessed: Sep. 10, 2026. [Online]. Available: https://scikit-learn.org/stable/auto_examples/model_selection/plot_nested_cross_validation_iris.html

[32] E. Tabassi, Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1, Jan. 2023, doi: 10.6028/NIST.AI.100-1.

[33] C. Autio et al., Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, NIST AI 600-1, Jul. 2024, doi: 10.6028/NIST.AI.600-1.